

JAVA

List & ArrayList

COLEÇÕES

Como o Java organiza coleções que crescem e encolhem em tempo de execução: a interface List, o funcionamento interno do ArrayList, a LinkedList como fila e pilha, Iterator, enum e autoboxing — tudo construído em torno da fila de reprodução de uma rádio online.

01 A Interface List e o ArrayList

02 Arrays vs. ArrayLists

03 Big O Notation & LinkedList

04 Iterator e ListIterator

05 O tipo enum

06 Autoboxing e Unboxing

JavaStudiesHub

A Interface List e o ArrayList

O que é a interface List, coding to an interface e como o ArrayList funciona por dentro

List é uma interface, não uma classe

Em Java, List não é algo que se instancia diretamente — é uma **interface**. Ela define um conjunto de comportamentos que qualquer implementação precisa oferecer, sem dizer como esses comportamentos devem ser implementados por dentro. Concretamente, List descreve uma coleção ordenada, acessível por índice inteiro, que aceita elementos duplicados e permite null.

Programar contra a interface List, em vez de amarrar o código a uma implementação concreta como ArrayList, tem uma vantagem prática: é possível trocar a implementação usada em um único lugar, sem alterar o restante do programa. Essa prática é conhecida como **coding to an interface** e é considerada uma boa prática de orientação a objetos.

```
List<String> filaDeReproducao = new ArrayList<>();
filaDeReproducao.add("Sunset Drive");
filaDeReproducao.add("Night Walk");
filaDeReproducao.remove("Sunset Drive");

System.out.println(filaDeReproducao.contains("Night Walk")); // true
System.out.println(filaDeReproducao.size()); // 1
```

Como o ArrayList funciona por dentro

O ArrayList guarda seus elementos em um array interno comum. Para evitar o custo de redimensionar o array a cada inserção, ele é criado com uma capacidade um pouco maior do que o número de elementos realmente usados — por padrão, a capacidade inicial é 10. Conforme elementos são adicionados, esse espaço vai sendo ocupado; quando ele se esgota, o Java cria internamente um array maior e copia todo o conteúdo para lá, de forma transparente para quem está usando a lista.

Essa estratégia é chamada de **crescimento amortizado**: em média, cada inserção tem custo constante $O(1)$, mesmo que, ocasionalmente, uma inserção específica precise disparar uma realocação $O(n)$. Ler ou substituir um elemento por índice (get/set) é sempre $O(1)$, porque o Java calcula diretamente o endereço de memória a partir do índice. Já inserir ou remover no meio da lista é $O(n)$, pois todos os elementos posteriores precisam ser deslocados uma posição.

```
ArrayList<String> playlist = new ArrayList<>(); // capacidade inicial: 10
playlist.add("Faixa A"); // índice 0
playlist.add(0, "Faixa Bônus"); // insere no início – desloca as demais
playlist.get(1); // "Faixa A" –  $O(1)$ 
playlist.set(0, "Faixa Intro"); // substitui "Faixa Bônus"
playlist.remove(0); // remove "Faixa Intro" – desloca elementos
```

Arrays vs. ArrayLists

Semelhanças, diferenças e quando escolher cada estrutura

Semelhantes por fora, diferentes por dentro

Arrays e ArrayLists parecem parecidos à primeira vista: os dois são ordenados por índice e aceitam elementos repetidos. Mas há diferenças importantes. A mais relevante: arrays aceitam tipos primitivos (int, double, boolean...), enquanto ArrayList só armazena tipos de referência (Integer, Double, Boolean...). O Java resolve isso automaticamente através do autoboxing, assunto do último capítulo desta apostila — mas vale entender essa limitação de origem.

Outra diferença central é o tamanho: um array tem tamanho fixo, definido no momento em que é criado — não é possível adicionar um elemento além do limite sem criar um array novo. O ArrayList, por sua vez, é redimensionável automaticamente, e ainda oferece métodos prontos como add(), remove(), contains() e sort() que um array simples não tem.

Aspecto	Array	ArrayList
Tamanho	Fixo, definido na criação	Redimensionável automaticamente
Tipos aceitos	Primitivos e referências	Só tipos de referência
Métodos utilitários	Só via java.util.Arrays	add, remove, contains, sort...
Desempenho de leitura	O(1), mínimo overhead	O(1), com overhead do objeto

QUANDO USAR CADA UM

Use array quando o tamanho é conhecido com antecedência e primitivos ou desempenho máximo importam. Use ArrayList para coleções que crescem e encolhem, com a conveniência dos métodos da interface List.

```
// Array – tamanho fixo, aceita primitivos
int[] duracoesEmSegundos = new int[5];

// ArrayList – redimensionável, só referências
ArrayList<Integer> duracoes = new ArrayList<>();
duracoes.add(180); duracoes.add(210); duracoes.add(195);
java.util.Collections.sort(duracoes); // ordenar com um método pronto
```

Big O Notation & LinkedList

Como cada estrutura usa a memória e quando a lista ligada vence o ArrayList

Memória contígua vs. nós ligados

Para escolher entre ArrayList e LinkedList, ajuda entender como cada uma organiza a memória. A notação Big O descreve como o tempo de execução de uma operação cresce em função do tamanho da entrada, no pior caso.

O ArrayList guarda seus elementos em um bloco contíguo de memória, o que permite calcular diretamente o endereço de qualquer posição: $\text{endereço} = \text{base} + \text{índice} \times \text{tamanho do elemento}$. Por isso, `get(índice)` é $O(1)$ — constante, independente do tamanho da lista.

A LinkedList funciona de um jeito bem diferente: cada elemento vive em um nó que guarda o valor, uma referência para o nó anterior e uma para o próximo. Os nós ficam espalhados pela memória, sem contiguidade nenhuma. Para alcançar a posição N, é preciso percorrer os nós um a um a partir de uma das pontas — o que torna `get(índice)` $O(n)$. Em compensação, inserir ou remover em uma ponta é $O(1)$: basta reajustar as referências dos nós vizinhos, sem deslocar nada.

Operação	ArrayList	LinkedList
<code>get(índice)</code>	$O(1)$	$O(n)$
<code>add</code> (no fim)	$O(1)$ amortizado	$O(1)$
<code>add</code> (no início/meio)	$O(n)$	$O(1)$ nas pontas
<code>remove(índice)</code>	$O(n)$	$O(1)$ nas pontas
<code>contains()</code>	$O(n)$	$O(n)$

LinkedList como Queue e como Stack

A LinkedList em Java implementa não só List, mas também Queue e Deque — o que a torna versátil o bastante para funcionar como lista comum, fila, pilha ou fila dupla, dependendo dos métodos escolhidos.

Como **Queue** (fila — FIFO, o primeiro que entra é o primeiro que sai): use `offer()` para adicionar ao final e `poll()` para remover do início. É exatamente o comportamento esperado de uma fila de reprodução — a próxima faixa a tocar é sempre a que está na frente da fila.

Como **Stack** (pilha — LIFO, o último que entra é o primeiro que sai): use `push()` para empilhar e `pop()` para desempilhar. Ideal para um histórico de "voltar", onde a última faixa ouvida é a primeira a reaparecer ao pedir para tocar a anterior.

```
// LinkedList como fila de reprodução (FIFO):
LinkedList<String> filaDeReproducao = new LinkedList<>();
filaDeReproducao.offer("Faixa A");
filaDeReproducao.offer("Faixa B");
filaDeReproducao.offer("Faixa C");
filaDeReproducao.poll(); // remove "Faixa A" – foi a primeira a entrar

// LinkedList como histórico de "voltar" (LIFO):
LinkedList<String> historico = new LinkedList<>();
historico.push("Faixa X");
historico.push("Faixa Y");
historico.pop(); // remove "Faixa Y" – foi a última a entrar
```

Iterator e ListIterator

Como remover elementos durante a iteração sem quebrar a coleção

O que um Iterator resolve

Um Iterator é um objeto que percorre uma coleção elemento a elemento, sem expor como essa coleção está organizada por dentro. Qualquer coleção que implemente Iterable — o que inclui List — pode fornecer um Iterator através do método iterator().

O Iterator mantém um cursor que começa posicionado antes do primeiro elemento. Cada chamada a next() avança o cursor e devolve o elemento atual; hasNext() informa se ainda existe um próximo elemento, e deve ser sempre chamado antes de next() para evitar um NoSuchElementException.

A grande vantagem do Iterator em relação ao for-each é permitir remoção segura durante a iteração, através de iterator.remove(). Tentar remover elementos de uma lista usando for-each durante a própria iteração lança ConcurrentModificationException — um erro comum entre quem está começando. O Iterator existe justamente para evitar esse problema.

```
List<String> filaDeReproducao = new ArrayList<>(
    List.of("Faixa A", "Faixa Pulada", "Faixa C", "Faixa D"));

Iterator<String> it = filaDeReproducao.iterator();
while (it.hasNext()) {
    String faixa = it.next();
    if (faixa.equals("Faixa Pulada")) {
        it.remove(); // seguro – sem ConcurrentModificationException!
    }
}
// filaDeReproducao agora: [Faixa A, Faixa C, Faixa D]
```

ListIterator — navegação nos dois sentidos

O ListIterator, disponível apenas para implementações de List, vai além: além de avançar com next(), ele também permite retroceder com previous(). É especialmente útil para navegar de forma bidirecional em uma fila de reprodução, alternando entre "próxima faixa" e "faixa anterior" sem precisar reconstruir nenhuma estrutura auxiliar.

```
List<String> playlist = new ArrayList<>(List.of("Faixa A", "Faixa B", "Faixa C"));
ListIterator<String> navegador = playlist.listIterator();

String proxima = navegador.next(); // "Faixa A"
String seguinte = navegador.next(); // "Faixa B"
String anterior = navegador.previous(); // "Faixa B" novamente – voltou uma posição
```

O tipo enum

Como declarar um conjunto fixo de valores nomeados, com segurança de tipo

Por que preferir enum a int ou String

O tipo enum permite declarar um conjunto fixo de constantes nomeadas e tipadas — em vez de representar categorias com inteiros soltos (0 para rock, 1 para pop...) ou Strings livres, que são fáceis de digitar errado e não oferecem nenhuma verificação em tempo de compilação. Um enum é, por baixo dos panos, uma classe especial, que também pode ter campos, construtores e métodos próprios.

Toda constante de um enum ganha automaticamente dois recursos: `ordinal()`, que devolve sua posição na declaração (a primeira é 0), e `name()`, que devolve o texto exato da constante como String. Enums também funcionam muito bem dentro de um `switch`, deixando o código mais legível e seguro do que comparar inteiros ou Strings manualmente.

```
enum Genero { ROCK, POP, JAZZ, ELETRONICA, MPB, CLASSICA }

Genero generoAtual = Genero.ELETRONICA;
System.out.println(generoAtual.ordinal()); // 3
System.out.println(generoAtual.name()); // "ELETRONICA"

// enum em switch – mais seguro que comparar String ou int:
switch (generoAtual) {
    case ROCK, ELETRONICA -> System.out.println("Sobe o volume!");
    case CLASSICA, JAZZ -> System.out.println("Ambiente calmo.");
    default -> System.out.println("Tocando " + generoAtual.name());
}
```

Autoboxing e Unboxing

A ponte automática entre tipos primitivos e as wrapper classes

Por que coleções não aceitam primitivos diretamente

O Java trabalha com dois sistemas de tipos: os primitivos (int, double, boolean, char...) e os tipos de referência (objetos). Coleções como ArrayList e LinkedList só conseguem armazenar objetos — nunca primitivos diretamente. Para viabilizar isso, o Java oferece as **wrapper classes**: Integer, Double, Boolean, Character, Long, Short, Byte e Float — uma para cada tipo primitivo.

Boxing é converter um primitivo para o wrapper correspondente; **unboxing** é o caminho inverso. Antes do Java 5, essa conversão era manual: Integer.valueOf(180) para boxing e caixa.intValue() para unboxing. Desde então, o compilador faz isso sozinho — daí o nome **autoboxing**.

A conveniência do autoboxing esconde um risco real: se um wrapper estiver com valor null e o código tentar fazer unboxing dele, o resultado é um NullPointerException em tempo de execução — um erro silencioso e difícil de rastrear até a causa. Vale sempre checar se o wrapper não é null antes de extrair o valor primitivo.

```
// Boxing manual (como era feito antes do Java 5):
Integer duracaoEmCaixa = Integer.valueOf(215);
int duracaoPrimitiva = duracaoEmCaixa.intValue();

// Autoboxing/unboxing automático (Java 5+):
ArrayList<Integer> duracoesEmSegundos = new ArrayList<>();
duracoesEmSegundos.add(215); // autoboxing: int → Integer
int primeira = duracoesEmSegundos.get(0); // unboxing: Integer → int

// Cuidado com null:
Integer duracaoDesconhecida = null;
int valor = duracaoDesconhecida; // NullPointerException em tempo de execução!
```

Primitivo	Wrapper class
int	Integer
double	Double
boolean	Boolean
char	Character
long	Long
float	Float
short	Short
byte	Byte

FECHAMENTO

Resumo da Apostila

Os conceitos essenciais desta apostila, em uma única tabela de consulta

Conceito	O que significa
List	Interface Java para coleções ordenadas por índice, com duplicatas e null permitidos.
ArrayList	Implementação de List com array interno redimensionável. get O(1), add/remove no meio O(n).
LinkedList	Lista duplamente ligada, também Queue e Deque. get O(n), add/remove nas pontas O(1).
Big O Notation	Descreve o crescimento do custo de uma operação. ArrayList favorece acesso aleatório; LinkedList favorece inserções nas pontas.
Iterator	Percorre uma coleção com segurança, permitindo remoção durante a iteração.
ListIterator	Iterator bidirecional, exclusivo de List — permite avançar e retroceder.
enum	Tipo para constantes fixas e tipadas, com ordinal(), name() e suporte nativo a switch.
Autoboxing / Unboxing	Conversão automática entre primitivo e wrapper (int ↔ Integer, etc.), feita pelo compilador.

JavaStudiesHub